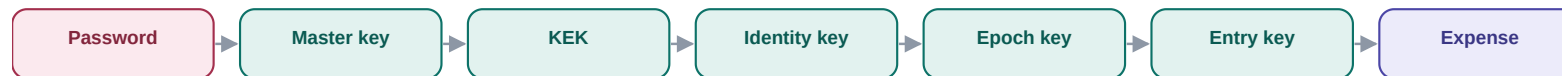


How the cryptography flows from a password to an expense

Six keys stand between the word somebody types and the row a database holds. Each one is opened by the one before it, and only the last one opens an expense. This document follows that chain end to end — where every key comes from, what binds it to the thing it opens, and what the server is holding while it happens.



What is on these pages

1. **The spine** password → expense, in one picture
2. **Account keys** register, log in, unlock, change password
3. **Group keys** epochs, wrapping, rotation, the keyring
4. **The envelope** how one expense is sealed
5. **Trust anchors** what stops the server handing over its own key
6. **Reference** every domain string and every AAD

Reading the colours

- ☐ **Secret, device only** — never sent anywhere, in any form
- ☐ **Ciphertext** — the server stores it and cannot open it
- ☐ **Public or server-held** — the metadata routing needs
- ☐ **Out of band** — a person, reading digits aloud

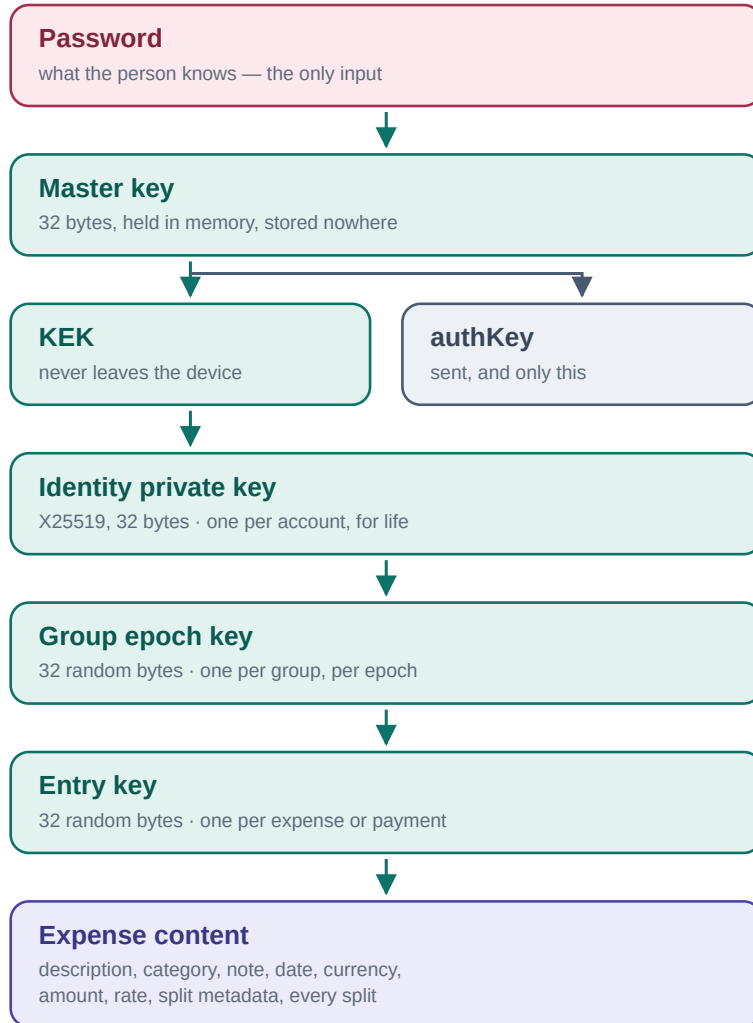
Primitives

Argon2id ([hash-wasm](#)) and X25519 ([@noble/curves](#)) come from libraries because WebCrypto does not portably provide them. AES-GCM, HKDF-SHA-256 and randomness are WebCrypto. Everything runs identically in the browser and in Node, so the tests exercise the same code paths the app does.

A database dump, a stolen backup or a curious operator reading tables gets ciphertext. What stays readable is the metadata the server must route on: who is in which group, entry counts, sizes and timestamps.

From a password to an expense

ON THE DEVICE



THE OPERATION

Argon2id

m = 19 MiB · t = 2 · p = 1 · 32-byte output
salt: 16 random bytes, made at registration

HKDF-SHA-256, twice

empty salt — the domain lives in the info string
spendapp/auth/v1 → authKey · spendapp/wrap/v1 → KEK

AES-GCM open, under the KEK

the identity private key, sealed at registration
users.wrapped_private_key

X25519 ECDH → HKDF → AES-GCM open

one wrap per member per epoch, ephemeral sender key
info: spendapp/wrap-key/v1|epk|recipient

AES-GCM open, under the epoch key

the wrapper rides on the entry's own row
aad: entrykey|expense|id|group|epoch

AES-GCM open, under the entry key

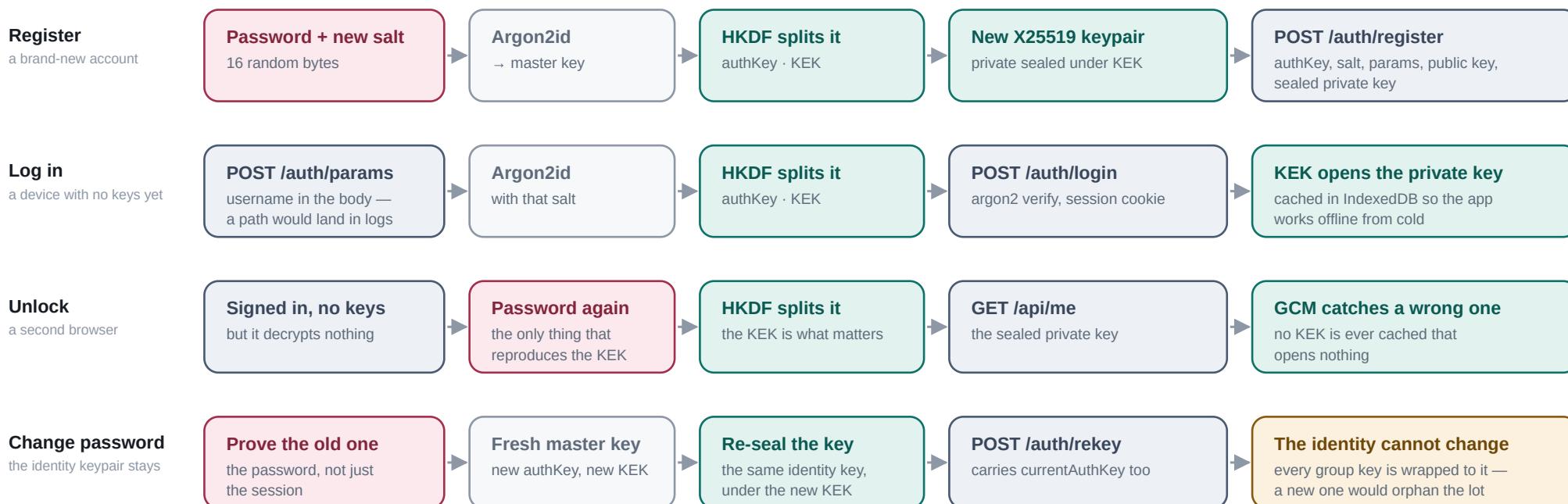
the tag is bound to the row it was written into
aad: expense|id|group|epoch

WHAT THE SERVER HOLDS OF IT

- **Nothing. Not now, not ever.**
The password is not sent, hashed or otherwise.
- **Nothing.**
It exists only while the app is unlocked.
- **argon2id(authKey) — a hash of a derived key, not of the password.** users.password_hash
- **Only the sealed copy, plus the**
X25519 public half it publishes to the group.
- **Only wraps: one blob per member,**
openable by that member's private key alone.
- **Only the wrapper, sealed under an epoch key** it does not have. key_iv / key_ct
- **iv + ct, padded to a size bucket,**
plus who wrote it, when, and in which group.

The rule the whole design turns on: the split at step two is one-way in both directions. The server is given **authKey** and stores only an Argon2id hash of it; the **KEK** — the half that actually unlocks anything — never leaves. Knowing either one tells you nothing about the other, so a database dump holds no value that opens the row below it.

Where the account's two keys come from, and what survives a password change



The users row, in full

password_hash	Argon2id of authKey — a hash of a derived key. The password itself was never in the request.
kdf_salt, kdf_params	Public by necessity: a client cannot re-derive without them. Stored per account, so raising the cost later does not invalidate old accounts.
public_key	The X25519 public half. Published to the group — this is what group keys get wrapped to.
wrapped_private_key	The private half, AES-GCM under the KEK. Opaque to the server for as long as it does not know the password.

Two details that are easy to miss

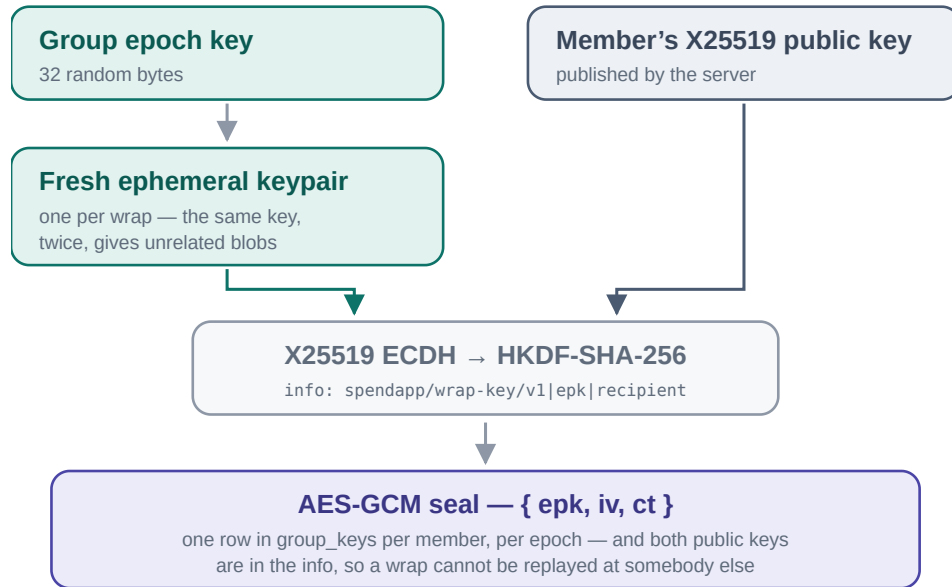
The username oracle. Login is a two-step handshake, so `/auth/params` would say whether an account exists. An unknown name instead gets a decoy: `HMAC-SHA256(server secret, name)`, stable across requests, shaped exactly like a real salt, and useless — no `authKey` derived from it will ever match. Unknown passwords are verified against a dummy hash so the timing matches too.

A forgotten password loses the data. There is no reset and deliberately no recovery code: the KEK exists nowhere but in the password. A *shared* group survives socially — another member re-wraps its keys to a fresh account, which is what the join flow already does. A group of one does not.

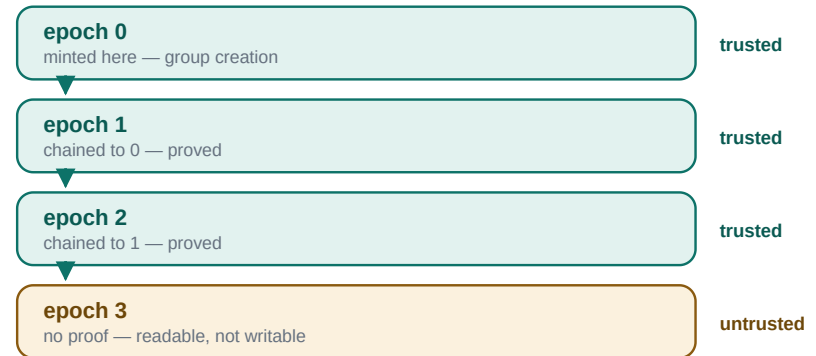
Where these keys sit once they are derived: the unwrapped KEK and identity keypair are cached in IndexedDB, so the app opens its own data offline from a cold start rather than demanding the password and an Argon2id run on every launch — which protects the data on the server, not on an unlocked stolen phone. The password itself lives inside one module for the length of one call, and nowhere else.

Epochs, wrapping, rotation, and the ring a member ends up holding

HOW AN EPOCH KEY REACHES A MEMBER



THE KEYRING A MEMBER HOLDS



Two rules, and they are different on purpose

Read with any key held — old entries and unproved epochs stay legible.
Write only under the highest trusted epoch — a server that invents an epoch wants new entries sealed under it, and refusing is what makes that pointless. Rotation is forward-only: it never claws back the past.

Three ways an epoch arrives

Minted. Epoch 0 is generated on the creator's device and travels inside the create-group request, so a group cannot exist without a key its creator can open.

Handed over. An approving member wraps the whole keyring to the joiner — wrapping only the current epoch would hand them a group that looks empty up to the last rotation. A *from-today* invite hands over one epoch instead, and that is what history scoping is.

Rotated. After a removal, a new epoch is minted and wrapped to whoever remains.

The chain proof

A rotation seals the new key *under the epoch it replaces*:

```

seal(keyn-1, keyn,
    aad = spendapp/key-chain/v1|group|n)
  
```

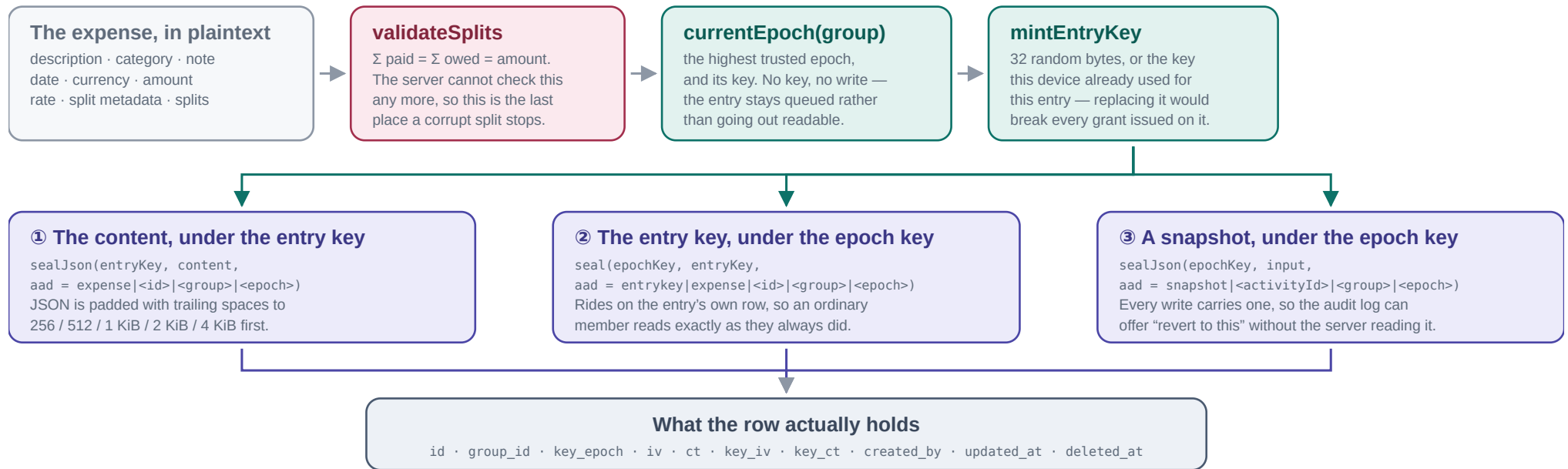
Producing that takes a member who already holds the old key. The server holds wraps it cannot open, so it cannot mint an epoch a client will write under. An *absent* proof is not a lie — epoch 0 has none, and neither does the floor of a scoped ring. A proof that is present and *wrong* is somebody claiming a lineage they do not have, and the epoch is dropped.

What rotation touches

The new epoch is wrapped to every remaining member, and the group's **name** is re-sealed under it — somebody admitted on this epoch alone has no other key to read it with. Minting is server-arbitrated (`mint: true`) so two admins removing people at once cannot both claim the same epoch number.

Entries already written do not move. Their epoch key still opens them, and the member who left keeps every key and every entry already on their device — re-encrypting the past would not change that.

What happens to one expense between the form and the database row



Everything sealed, and what seals it

WHAT	SEALED UNDER	BOUND TO (AAD)
Expense / payment content	its own entry key	<code>expense id group epoch</code>
The entry key itself	the epoch key	<code>entrykey type id group epoch</code>
Version snapshot	the epoch key	<code>snapshot activityId group epoch</code>
Comment	the epoch key	<code>comment id group epoch</code>
Receipt image	the epoch key	<code>attachment id group epoch</code>
Group name	the <i>newest</i> epoch key	<code>groupname group epoch</code>

The AAD is authenticated but not encrypted, so it costs nothing to store and a blob lifted from one row into another simply will not open. Two versions of one expense would otherwise share an AAD and could be swapped — which is why a snapshot is bound to its own log row, not to the entry.

Three consequences of doing it this way

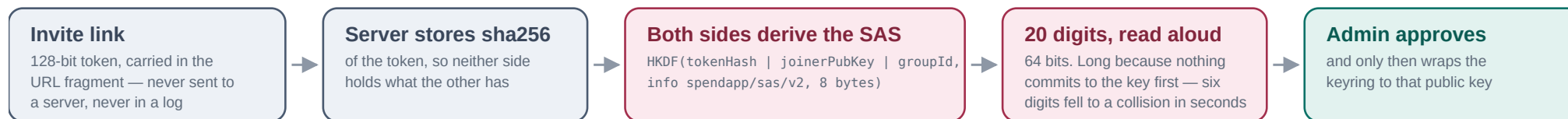
Length is a leak, so it is bucketed. AES-GCM does not hide size: the exact byte count of a row separates a two-way split from a ten-way one and a one-word note from a paragraph. Padding is trailing spaces, which `JSON.parse` already ignores, so nothing stored had to move when it was introduced. It stops at 4 KiB, where a record gives away only that it is large.

A receipt is a file, not a row. The IV rides in front of the ciphertext instead of in a column, so the two can never be separated or mispaired. The image type is sniffed on the device when it comes back — the server never sees enough to know it was ever an image.

The outbox re-seals on the way out. A device that was offline when the group rotated has mutations sealed under the old epoch. Re-sealing at upload time is byte-level: the plaintext is opened and sealed again exactly as it is, only the AAD changes, and the result is opened once more and compared before the queued copy — the only copy — is replaced.

What stops the server from handing over a key of its own

A · JOINING — THE DIGITS BOTH SIDES READ ALOUD



It authenticates the joiner's public key to the admin — the direction that stops the wrong person being let in. It says nothing about the keys travelling back.

B · AFTERWARDS — WHAT THIS ACCOUNT ITSELF RECORDED



On a second device, or one whose cache was cleared: a delivered key that contradicts this account's own commitment is refused outright — not merely held untrusted — and the member is told which epoch.

Why a commitment beats a wrap

A wrap arrives sealed to a public key *the server itself publishes*, so nothing in it says a member produced it. A commitment is sealed under a key only this account's devices can derive. So it is the one thing in a sync payload that is a statement *by us*, made at a time we held the real key, about what that key was.

Derived from the identity private key rather than the KEK on purpose: a new password means a new KEK, and every commitment this account had ever written would become unopenable and unreplaceable — the anchor silently gone on the next fresh device.

Two numbers members compare by voice

Epoch digits. Over the newest epoch held, named alongside the epoch so two people compare like with like rather than reading a mismatch out of one being a sync behind. Every current member holds the newest epoch by construction, so this number is comparable across the whole group.

Keyring digits. Strictly stronger where they apply — they cover every key a hand-over delivered, including the oldest held, which has no predecessor to chain to. But two honest rings differ whenever history differs, so these are offered only when the server reports that every member holds every epoch. It can lie about that; the worst it buys is a false alarm.

The gap that is left, stated plainly

The very first key a brand-new account is given is still taken on trust. The ring is empty, so there is no predecessor to chain to and no commitment of this account's to contradict. A server substituting the key at that moment reads everything the member writes afterwards.

Reading the digits aloud, once, is the only thing standing in the way — and every delivery after that is anchored, by the chain, by the commitment, or by both.

And it protects the server, not the phone. Keys are cached unwrapped in IndexedDB so the app works offline from a cold start.

The three anchors, in the order they are checked

1 · This account's own commitment. A delivered key whose fingerprint disagrees is *refused* — no honest party can produce one — and accepted without needing a chain, because our own past word is a better anchor than a proof.

2 · The chain proof. Present and correct: trusted. Present and wrong: the epoch is *dropped*. Absent: held, readable, but never written under — epoch 0 and the floor of a scoped ring legitimately have none.

3 · The hand-over itself. The one delivery trusted unconditionally, because the ring is empty and there is nothing to check it against. This is where the digits read aloud, and nothing else, are doing the work.

Every domain string, every binding, and what is left readable

Domain separation — every HKDF info string

The salt is empty everywhere: the input is already a high-entropy Argon2id output or an ECDH secret, and the separation lives entirely here. No two uses of the same key material can collide.

<code>spendapp/auth/v1</code>	master key → authKey
<code>spendapp/wrap/v1</code>	master key → KEK
<code>spendapp/wrap-key/v1 epk recipient</code>	ECDH secret → the key that wraps a group or entry key
<code>spendapp/key-fingerprint/v1</code>	epoch key → a one-way name for it, safe to store
<code>spendapp/key-commitment-key/v1</code>	identity private key → the commitment key
<code>spendapp/sas/v2</code>	token hash + joiner key + group → 20 join digits
<code>spendapp/epoch-sas/v1</code>	group + epoch + fingerprint → 20 digits
<code>spendapp/keyring-sas/v1</code>	group + the whole ring → 20 digits

Every AAD, and what it prevents

<code>expense id group epoch</code>	A blob cannot be lifted from one row and replayed into another.
<code>payment id group epoch</code>	
<code>entrykey type id group epoch</code>	Where “this entry belongs to that epoch” is actually authenticated.
<code>snapshot activityId group epoch</code>	Two versions of one expense cannot be swapped for each other.
<code>comment id group epoch</code>	A comment cannot be moved to another entry.
<code>attachment id group epoch</code>	A receipt cannot be moved to another expense.
<code>groupname group epoch</code>	Bound to the group and epoch only — so renaming is not a re-seal of everything.
<code>spendapp/key-chain/v1 group epoch</code>	A rotation proof cannot be replayed at a different epoch.
<code>spendapp/key-commitment/v1 group epoch user</code>	The server cannot move a commitment between epochs or users.

What a database dump contains

OPAQUE

- Every expense and payment: amounts, currencies, descriptions, notes, categories, dates, and who owes whom
- Every comment
- Every receipt image
- Every group name
- Every version snapshot in the audit trail
- Every group key, entry key and identity private key

READABLE — THE METADATA ROUTING NEEDS

- Who is in which group, and who is an admin
- How many entries exist, roughly how large, and when each was written or changed
- Which account wrote which row
- Display names, usernames, session and push-subscription records
- Which epoch each row belongs to, and who holds a wrap for it

The release check is one line, and it is the only test of the actual claim:

```
mysqldump spendapp > /tmp/check.sql && grep -i 'a description you know is in there' /tmp/check.sql
```

It must find nothing. The server test suite pins the sealed tables to explicit column lists, so a plaintext column reappearing fails in CI rather than in the dump.

What this costs, deliberately

The server validates no money. It cannot see a split, so a modified client can write a corrupt entry into a shared group. Clients check on read and refuse it, and the group is told which entry and who wrote it.

No server-side search, reporting or aggregation, permanently.

Notifications fan out to everybody. Sending only to the people an entry names would mean telling the server who they are — the participant graph, which is worth more than the notification it would buy. So every member's device is woken by every entry and filters on arrival.

Download my data is built on the device. The server holds ciphertext, so it could never produce a readable copy, and an archive of ciphertext would not be portable in any useful sense.